

History of C++

◆ Origin of C++

- C++ was developed by Bjarne Stroustrup
- Developed at **AT&T Bell Laboratories, USA**
- Development started in **1979**
- Initially called “**C with Classes**”

📌 Goal:

To add **Object-Oriented Programming features** to the C language without losing its efficiency.

◆ Why C++ Was Developed?

C language:

- Supports **procedural programming**
- Not suitable for large & complex applications
- Lacks features like:
 - Classes
 - Objects
 - Inheritance
 - Polymorphism

👉 Hence, C++ was created to overcome these limitations.

◆ Evolution of C++

Year	Version / Name	Description
1979	C with Classes	Added classes & basic OOP
1983	C++	Name changed to C++
1985	C++ 1.0	First commercial release
1990	C++ 2.0	Added inheritance, polymorphism
1998	C++98	First ISO standard
2003	C++03	Minor improvements
2011	C++11	Major update (auto, lambda, STL improvements)
2014	C++14	Enhancements
2017	C++17	Performance improvements
2020	C++20	Concepts, ranges
2023	C++23	Latest refinements

◆ **Meaning of “C++”**

- **C++ means “increment of C”**
- ++ is the increment operator
- Symbolizes **enhancement of C language**

◆ **Features Added in C++**

Compared to C, C++ introduced:

- **Classes & Objects**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Data abstraction**
- **Function & Operator Overloading**

◆ **Design Philosophy**

Bjarne Stroustrup designed C++ to:

- **Be fast and efficient**
- **Be backward compatible with C**
- **Support multiple programming paradigms**
 - Procedural
 - Object-Oriented
 - Generic programming

◆ **Popularity & Usage**

C++ is widely used in:

- **Operating systems**
- **Game development**
- **Embedded systems**
- **Compilers**
- **High-performance applications**

◆ **One-Line Exam Answer**

C++ is an object-oriented programming language developed by Bjarne Stroustrup at AT&T Bell Labs in 1979 as an extension of the C language.

Tools to Perform C++ Programs

To write, compile, and execute C++ programs, we need the following tools:

1. Text Editor

Used to **write C++ source code** (.cpp file).

Examples:

- Notepad
- Notepad++
- VS Code (recommended)
- Sublime Text

✦ *Function:* Writing and editing C++ programs

2. Compiler

Converts **C++ source code** into **machine code**.

Popular C++ Compilers:

- **GCC (GNU Compiler Collection)**
- Clang
- Microsoft Visual C++ (MSVC)

✦ *Function:* Checks syntax errors and generates executable file

3. Integrated Development Environment (IDE)

An IDE combines:

- Editor
- Compiler
- Debugger

Popular C++ IDEs:

IDE	Platform	Remarks
Dev-C++	Windows	Beginner-friendly
Code::Blocks	Cross-platform	Widely used in academics
Visual Studio	Windows	Industry-level
VS Code	Cross-platform	Lightweight & modern
CLion	Cross-platform	Professional (Paid)

✦ *Function:* Complete environment for C++ development

4. Debugger

Used to **find and fix errors** in programs.

Examples:

- GDB
- LLDB
- Visual Studio Debugger

✦ *Function:* Step-by-step execution and error detection

5. Command Line / Terminal

Used to compile and run C++ programs manually.

Example:

```
g++ program.cpp  
./a.out
```

✦ *Function:* Program execution and compilation control

6. Online C++ Compilers (For Practice)

Useful when compiler is not installed.

Examples:

- Programiz
- GeeksforGeeks IDE
- OnlineGDB
- Replit

✦ *Function:* Instant execution & learning



Steps to Execute a C++ Program

1. Write code using **editor**
 2. Save file as .cpp
 3. Compile using **compiler**
 4. Fix errors (if any)
 5. Run the program
 6. View output
-



Exam-Oriented Definition

Tools used to perform C++ programs include a text editor, compiler, IDE, debugger, and execution environment.



Recommendation for Students

For beginners:

- **Dev-C++ or Code::Blocks**
For modern learning:
- **VS Code + GCC**

First C++ Program: Compilation & Execution Process

First C++ Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

Is C++ Compiled or Interpreted?

👉 C++ is a **compiled language**, not interpreted.

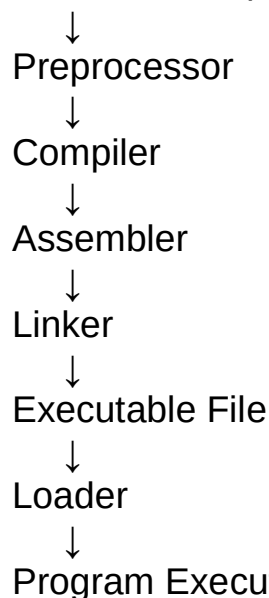
- **Compiled** → Entire program converted to machine code before execution
- **Interpreter** → Executes code line-by-line (like Python)

✦ *Important clarification for students:*

C++ does **not** use an interpreter, but **debuggers may simulate step-by-step execution**, which often causes confusion.

C++ Program Execution Flow (High-Level)

Source Code (.cpp)



Step-by-Step Technical Explanation

◆ Step 1: Writing Source Code

- Written in **text editor / IDE**
- Saved as `program.cpp`

✦ Contains:

- Preprocessor directives
 - C++ statements
 - Functions and logic
-

◆ Step 2: Preprocessing

Handled by **preprocessor**.

Tasks:

- Processes `#include <iostream>`
- Replaces macros
- Removes comments

✦ Output:

➡ Expanded source code (not visible to user)

Example:

```
#include <iostream>
```

is replaced by the entire content of the `iostream` header.

◆ Step 3: Compilation

Handled by **C++ Compiler (g++)**

Compiler tasks:

- Syntax checking
- Type checking
- Converts code to **object code**

✚ If error occurs:

error: expected ';' before 'return'

✚ Output:

program.o (object file)

◆ Step 4: Assembling

- Converts object code to **machine-level instructions**
 - Mostly automatic, not visible
-

◆ Step 5: Linking

Handled by **Linker**

Linker tasks:

- Links object file with libraries
- Resolves functions like:
- cout

(defined in iostream library)

✚ If missing:

undefined reference to 'cout'

✚ Output:

- a.out (Linux)
 - program.exe (Windows)
-

◆ Step 6: Loading

Handled by **Loader (OS responsibility)**

- Loads executable into RAM
 - Assigns memory
 - Starts execution from main()
-

◆ Step 7: Program Execution

- CPU executes machine instructions
- Output appears on screen

Hello, World!

Role of Compiler in C++

✦ Compiler:

- Translates **entire program at once**
- Faster execution
- Errors shown before execution

Example:

g++ hello.cpp

Role of Debugger (Important Concept)

Debugger is **NOT** part of compilation.

Debugger Functions:

- Execute program **step-by-step**
- Set breakpoints
- Inspect variable values
- Detect runtime errors

✦ Example Debugging Actions:

- Pause at main()
- Watch value of variables
- Find logical errors

💡 Common Student Confusion:

Debugger feels like an interpreter, but it is **not interpreting code**, only **controlling execution of compiled code**.

Why C++ Has No Interpreter?

Feature	C++	Python
Execution	Compiled	Interpreted
Speed	Very fast	Slower
Error Detection	Before execution	During execution
Debugging	Via debugger	Via interpreter

Command-Line Demonstration (Optional in Lab)

```
g++ hello.cpp -o hello
./hello
```